

CHALMERS



Syntes av trum- och percussionljud med hjälp av frekvensmodulering

Erik Larsson

EX048/2000
Institutionen för Signaler och System
November 2000

*Ett stort tack riktas till samtliga på Elektron
ESI AB, som välvilligt bidragit med tips och
synpunkter under arbetets gång, samt till
min handledare på Chalmers; Johannes Win-
tenby.*



Abstract/sammanfattning

Within this project, a basic FM (frequency modulation) algorithm for the synthesis of drum and percussion sounds was developed. The algorithm is based on the work of J. M. Chowning, who pioneered the use of FM for sound generation. A number of adaptations were made to the algorithm, to make it more suitable for different specific instruments. These include bassdrum, snaredrum, toms, cowbell, handclap and cymbals. The algorithms were finally implemented in Motorola DSP56303 assembly language. The resulting code will be used in the "Machinedrum"; a drum synthesizer developed by Elektron ESI AB in Göteborg, Sweden.

Innehållsförteckning

Abstract/sammanfattning	3
Innehållsförteckning	4
1 Introduktion	5
1.1 Bakgrund	5
1.2 Arbetsuppgiften	5
1.3 Allmänt om FM	6
2 FM-syntes, teori	9
2.1 Grunderna	9
2.2 Reflekterade sidofrekvenser, harmoniskt spektrum	10
2.3 Frekvens- eller fasmodulering: begreppsförvirring	10
2.4 Modulatorfasens betydelse	11
2.5 Utvidgningar av grundkonceptet	12
2.6 Återkopplad FM	13
3 Konstruktion av algoritmer	15
3.1 FM Gen	15
3.2 Utvecklingsarbetet	15
4 Resultat	18
4.1 Bastrumma	18
4.2 Virveltrumma	19
4.3 Pukor	20
4.4 Handklapp	21
4.5 Kantslag	22
4.6 Koklocka	23
4.7 Cymbaler (öppen och stängd hihat, ride, crash)	24
5 Implementering	27
5.1 Hårdvaran	27
5.2 DSP-programmering	27
5.3 Coldfire-programmering	27
5.4 Brusgeneratorn	27
5.5 Oscillatorer	28
5.6 Högpassfiltret	28
6 Slutsats och kommentar	29
Referenser	30

1 Introduktion

1.1 Bakgrund

Detta examensarbete är utfört på företaget Elektron ESI AB i Göteborg. Grunden till Elektron lades under projektkursen i digital konstruktion på Chalmers D-linje, då några chalmester kom på idén att utveckla en synth kring det ljudchip som satt i den gamla hemdatorn Commodore 64; SID6581. Resultatet blev SidStation, ett väldigt udda musikinstrument som ger vanliga, "MIDI-orienterade", musiker tillgång till det karakteristiska SID-ljudet. SidStation har fått väldigt bra recensioner både i nationell och internationell fackpress. Efter denna framgång var det dags att gå vidare och konstruktionen av nästa produkt, Machinedrum, påbörjades.

Machinedrum kan beskrivas som en avancerad trummaskin eller snarare trum-synthesizer, eftersom möjligheterna att själv påverka trumljudens karaktär är betydligt större än vad de varit i tidigare maskiner. Trummaskiner lever f.ö. en tynande tillvaro anno 2000 - förvånansvärt få rena trummaskiner tillverkas idag. En del av de klassiska¹ analoga apparaternas ljud lever kvar i form av samplingar i diverse synthesizers och "grooveboxar"², men få syntetiserar egna trumljud.

Machinedrum är en DSP-baserad, öppen arkitektur, där all ljudgenerering sker i mjukvara. Detta gör att systemet kan innehålla flera vitt skilda syntesmetoder och även kompletteras med nya i framtida versioner av operativsystemet. Min uppgift har varit att skapa DSP-algoritmer för generering av trumljud med hjälp av FM (frekvensmodulering), och dessa är tänkta att vara en utav flera andra ljudmodeller i maskinen. Jag har givits väldigt fria händer att själva utforma ljudalgoritmerna - enda kravet har egentligen varit att syntesen på något sätt skall bygga på FM. Handledare på Chalmers har varit Johannes Wintenby vid Institutionen för Signaler och System. På Elektron handledde Daniel Hansson.

1.2 Arbetsuppgiften

Följande rader är direkt tagna ur uppgiftsbeskrivningen från Elektron:

Syftet med exjobbet är att utreda potentialen hos frekvensmodulationsalgoritmen (FM-syntes) för att framställa perkussiva ljud på syntetisk väg. Algoritmens möjligheter inom detta område är hittills oprövad. Det kommer att krävas en anpassning av algoritmen för att frambringa önskvärd karaktär.

Arbetet skall vila på en vetenskaplig grund där en grundlig utredning av algoritmens egenskaper krävs samt en djupgående förståelse. De ur arbetet föreslagna tillämpningarna på algoritmen skall implementeras på en DSP-plattform (digital signalprocessor) med realtidskrav.

Examensarbetet innefattar således följande delsteg:

- FM-algoritmens möjligheter att framställa perkussiva ljud skall utredas.

-
1. Kända och välkända ljud kommer från maskiner som Rolands TR-808 och TR-909 - idag dyra och eftersökta på begagnatmarknaden.
 2. Mer eller mindre leksaksbetonade apparater som medger enkel framställning av technomusik.

- Resultatet av denna utredning skall ligga till grund för en implementering på en DSP-plattform.
- Utvärdering av arbetets resultat och funktionalitet.

Programmering sker lämpligen i programspråket C, med slutlig implementering i assembler på DSP för högsta effektivitet.

Uppgiftens omfattning gör detta arbete lämpligt för en person.

Mål:

- De algoritmer som används ska i möjligaste mån undvika hög beräkningskomplexitet
- Ljud av följande typer bör kunna genereras: Bastrumma, virveltrumma, hihat, cymbal samt pukor.
- En viktig aspekt på de genererade ljuden är att de har en nyskapande karaktär.
- Programkoden skall vara utförligt kommenterad.
- Arbetet skall vara noggrant dokumenterat och även de svårigheter och hinder som har upptäckts under projektets gång skall vara tydligt beskrivna.
- Dessutom bör rapporten innefatta ett avsnitt om eventuella andra tillämpningar av den undersökta algoritmen.

-

Målet har inte varit att skapa naturtrogna trumimitationer, utan nyskapande och karaktärsstarka ljud där man ändå kan höra vilken typ av perkussionsinstrument som avses (åtminstone i deras standardinställningar). Följande ljud ingår i ett komplett Machinedrum-kit:

- Bastrumma
- Virveltrumma
- Hög puka
- Mellanpuka
- Låg puka
- Handklapp
- Kantslag
- Koklocka
- Stängd hihat
- Öppen hihat
- Ridecymbal
- Crashcymbal

På maskinen finns åtta rattar för styrning av ljudens karaktär. Till varje trummodell skall därför finnas högst åtta parametrar som kan kontrolleras av användaren. Att bestämma dessa kontroller har också varit en viktig del av arbetet - ljuden skall vara enkla att styra och förändra.

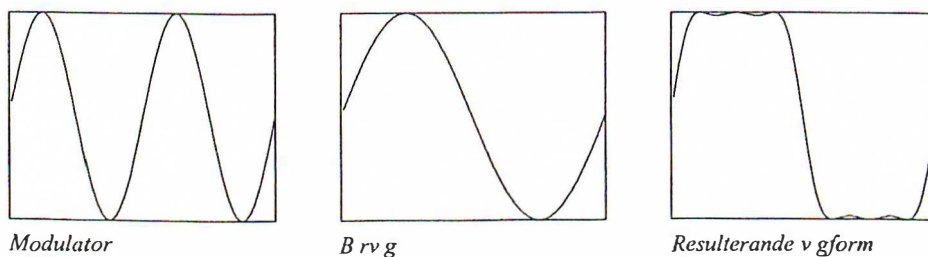
1.3 Allmänt om FM

Frekvensmodulering (FM) är sedan länge en välkänd teknik i radiosammanhang. Frekvensen för en oscillator (bärvågen) tillåts avvika från ett fast medelvärde. Mängden avvikelse styrs av en annan signal (modulatorn) och denna

signal kan sedan återskapas i mottagaren. Typiskt är bärvågens frekvens flera tiopotenser större än modulators (fig. 1.1). Motsvarigheten i musiksammanhang kallas vibrato: långsamma variationer av en tons frekvens, till exempel hos en operasångare. Om däremot modulatorens frekvens är i samma storleksordning som bärvågens (och inom det hörbara spektrat) blir frekvensavvikelserna för snabba för att kunna uppfattas som ett vibrato. Istället ändrar tonen karaktär (fig. 1.2); modulatorens skapar ett spektrum av övertoner och undertoner kring bärvågen - en ny vågform uppstår.



Figur 1.1: Frekvensmodulerad våg



Figur 1.2: FM-syntes skapar en ny vågform - här något som påminner om en fyrkantsvåg

Den första att uppmärksamma denna metod att syntetisera ljud var Dr. John M. Chowning vid Stanford, som 1973 publicerade den första artikeln i ämnet [1]. FM-ljudet skiljde sig väsentligt från vad som tidigare kunnat åstadkommas och de FM-synthar¹ som så småningom dök upp var på sin tid banbrytande. De var inte nödvändigtvis bättre - ljudet av klassisk subtraktiv syntes² är till exempel betydligt mer eftersökt idag - men liknade ändå mer riktiga instrument (även om det ofta krävs lite god vilja för att höra likheten).

En viktig del av FM-ljudet är det "dynamiska spektrum" som enkelt kan åstadkommas: modulatorens amplitud styr energins fördelning i spektrat och därför låter man ofta denna amplitud förändras under tiden ljudet spelas. På så sätt kan ett klockljud börja med ett komplext spektrum (stor modulatoramplitud) för att så småningom klinga ut mot en ren sinuston (ingen modulering alls).

Med FM kan invecklade vågformer skapas med små medel - två sinusoscillatorer räcker långt. Tekniken har därför ända fram till våra dagar varit populär i olika sorters budgetsynthar, till exempel de som finns inbyggda i TV-spel och

1. Mest känd är Yamaha:s DX7 (fig. 1.3), introducerad 1983, som definitivt färgade mycket av 80-talets musik. Det karaktäristiska elpianoljud som går att framställa med DX7 (tänkt att efterlikna det halvakustiska Fender Rhodes) kan höras i ett oräkneligt antal låtar, och används fortfarande flitigt.
2. Den vanligaste syntesmetoden än i dag; man utgår från en vågform med mycket övertoner (till exempel fyrkantsvåg, sågtagandsvåg eller en sampling) och skär sedan bort frekvenskomponenter med hjälp av filter.

enkla PC-ljudkort. Eftersom FM fungerar bäst med stabila, noggranna oscillatorer är de flesta implementationer digitala. Kapitel 2 förklarar FM-syntes mer ingående.



Figur 1.3: Yamaha DX7

2 FM-syntes, teori

2.1 Grunderna

FM kräver i sin enklaste form två sinusoscillatorer, eller *operatorer* som det brukar kallas i FM-sammanhang. Den ena fungerar då som bärvåg och den andra som modulator. En sådan frekvensmodulerad signal beskrivs av ekvationen

$$x(t) = A \sin(\alpha t + I \sin \beta t) \quad (2.1)$$

där

x = den modulerade bärvågens momentana amplitud

A = den modulerade bärvågens maximala amplitud

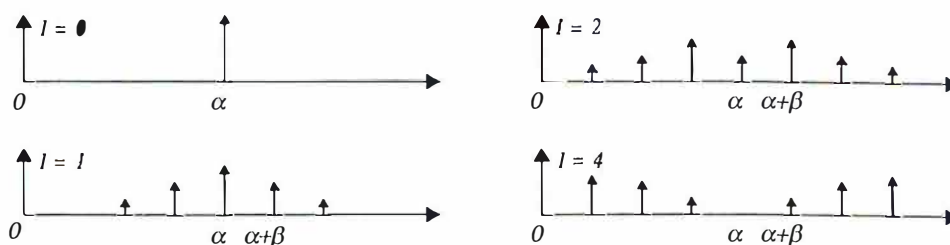
α = bärvågens vinkelfrekvens i rad/s

β = modulatorns vinkelfrekvens i rad/s

I = modulationsindex

t = tiden

När $I = 0$ existerar ingen frekvensavvikelse och ingen modulering förekommer. När I däremot är större (eller mindre) än noll uppstår nya frekvenskomponenter, symmetriskt fördelade över och under bärvågen. Dessa ligger med ett intervall som direkt motsvarar modulatorfrekvensen. Allt eftersom I ökar uppstår fler sidofrekvenser och energin förflyttas alltmer bort från bärvågen (fig. 2.1).



Figur 2.1: Bandbredden ökar med modulationsindex (I)

Man kan visa [2] att när $x(t)$ uttrycks i form av en fourierserie bestäms koefficienterna av besselfunktioner, $J_n(I)$, på följande sätt:

$$x(t) = \sum_{n=-\infty}^{\infty} J_{|n|}(I) \sin\left(\alpha t + n\beta t + n\frac{\pi}{2} - |n|\frac{\pi}{2}\right) \quad (2.2)$$

Utveckling av ovanstående uttryck ger

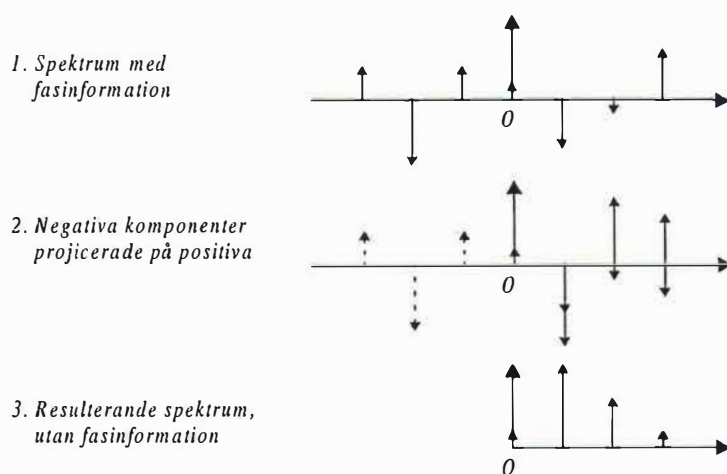
$$\begin{aligned} x(t) = A \{ & J_0(I) \sin \alpha t \\ & + J_1(I) \sin(\alpha t + \beta t) - J_1(I) \sin(\alpha t - \beta t) \\ & + J_2(I) \sin(\alpha t + 2\beta t) + J_2(I) \sin(\alpha t - 2\beta t) \\ & + J_3(I) \sin(\alpha t + 3\beta t) - J_3(I) \sin(\alpha t - 3\beta t) \dots \} \end{aligned} \quad (2.3)$$

Man kan se i ekvation 2.3 att vissa frekvenskomponenter får *negativa* koefficienter. Även besselfunktionerna är för vissa kombinationer av n och I negativa. Detta motsvarar en 180 graders fäsförskjutning och borde egentligen inte spela någon större roll - örat är dåligt på att urskilja fasskillnader. På det vis FM används i ljudsammanhang blir det dock viktigt: en modulator i samma storleksordning som bärvågen skapar ofta sidofrekvenser som reflekteras kring 0 Hz-axeln för att sedan interferera med positiva komponenter.

2.2 Reflekterade sidofrekvenser, harmoniskt spektrum

Vid vissa kombinationer av I , α och β uppstår komponenter med negativ frekvens (när $\alpha - n\beta < 0^1$). När de reflekteras kring 0 Hz-axeln blandas de med de positiva frekvenskomponenterna och deras fas är därför viktig - två lika komponenter i motfas släcker till exempel ut varandra helt (fig. 2.2). När α och β förhåller sig till varandra som två heltal, dvs. när α/β är ett rationellt tal, skapas ett harmoniskt spektrum². Eftersom detta är tilltalande för örat är det något man ofta eftersträvar när man syntetiserar ljud med FM. Detta är också förutsättningen för att negativa (reflekterade) frekvenskomponenter skall interferera med positiva. Observera att grundtonen inte nödvändigtvis behöver utgöras av bärvågen - en sidofrekvens, reflekterad eller ej, kan mycket väl ta över denna roll.

Fenomenet med reflekterade sidofrekvenser påverkar i stor utsträckning ljudets klangfärg - ofta är *samtliga* frekvenser under bärvågen negativa (när $\beta > \alpha$) och reflekteras därmed.



Figur 2.2: Negativa och positiva frekvenser bildar tillsammans spektrat (3).

2.3 Frekvens- eller fasmodulering: begreppsförvirring

I återstoden av denna rapport används begreppet *frekvensmodulering* med innebörden att man momentant förändrar en oscillators vinkelfrekvens med hjälp

1. Teoretiskt finns förstås alltid sådana frekvenserkomponenter närvarande, men vid små I blir besselfunktionernas amplitud med ökande n snart försumbart liten.
2. I ett hamoniskt spektrum har övertonerna frekvenser som är jämna multipler av grundtonen.

av en modulatorfunktion av något slag. Vid en närmare granskning av FM-ekvationen (2.1) ser man att α är hela tiden konstant medan $I\sin\beta t$ adderar en periodiskt varierande *fasförskjutning* till uttrycket. Detta är dock *nästan* samma sak som frekvensmodulering (enligt vår definition) *förutsatt att modulatorens är sinusformad*, vilket vi snart skall se. Om vi deriverar argumentet i ekvation 2.1 får vi bärvågens momentana vinkelfrekvens:

$$\frac{d}{dt}[\alpha t + I\sin\beta t] = \alpha + I\beta\cos\beta t \quad (2.4)$$

Vi ser här att ekvation 2.1 med en sinusformad modulator egentligen motsvarar frekvensmodulering med cosinus, och det ger inte riktigt samma resultat (se kapitel 2.4). Vid praktisk implementering av FM används ofta oscillatorer i form av sinustabeller. Det är då enkelt att åstadkomma FM där man verkligen styr den momentana frekvensen, dvs. steglängden i tabellen. För att få det förväntade resultatet måste alltså modulatorens i så fall bestå av en cosinusfunktion, dvs. vara fasförskjuten +90 grader relativt bärvågen.

I Chownings artikel från 1973 används FM-formeln (2.1) sida vid sida med enkla "kopplingscheman" som ger ett intryck av att det verkligen är den momentana *frekvensen* som moduleras, utan att något nämns om denna viktiga skillnad. Det samma gäller en mängd efterföljande artiklar. Detta har lett till en del förvirring genom åren - implementationer som har förutsatts vara likvärdiga har gett olika resultat. Inte förrän 1992 gjordes ett försök att reda ut begreppen och skapa en "FM-standard" (F. Holm, "Understanding FM Implementations: A Call for Common Standards" [3]).

Frekvensmodulering enligt vår definition beskrivs i med följande ekvation:

$$x(t) = \sin(\alpha t + I\int m(\beta t)) \quad (2.5)$$

där $m(\beta t)$ är modulatorfunktionen. Om vi deriverar argumentet ser vi att detta stämmer:

$$\frac{d}{dt}[\alpha t + I\int m(\beta t)] = \alpha + I\beta m(\beta t) \quad (2.6)$$

Om vi frekvensmodulerar med $m(t) = \cos t$ har vi alltså uppnått samma resultat som Chowning gjorde med grundformeln (2.1). Genom att fasförskjuta modulatorens har de två metoderna gjorts likvärdiga, men det är viktigt att komma ihåg att detta endast fungerar när modulatorens är sinusformad - om $m(t)$ till exempel innehåller en DC-komponent förflyttas hela grundtonen. Det kan vara värt att notera att FM, enligt vår definition, med cosinusmodulator inte ger *exakt* det väntade resultatet från ekvation 2.1 när den implementeras digitalt - detta på grund av att stegningen genom tabelloscillatorn endast approximerar en integral.

2.4 Modulatorfasens betydelse

Vi har ovan sett att det finns två huvudsakliga metoder att implementera FM - enligt Chownings formel (2.1), eller den mer intuitiva lösningen med "oscillatorer", där den momentana frekvensen moduleras. Båda används flitigt i olika sammanhang. Vi har också sett att dessa kan göras likvärdiga genom att justera

fasförhållandet mellan bärvåg och modulator, men har ännu inte sett vilken effekt denna fasparameter har. Det kommer att visa sig att ljudet påverkas på ett inte helt obetydligt vis, och det är därför lämpligt att utöka ekvation 2.2 med en fasparameter, ϕ_m . Detta ger följande, mer generella, ekvation:

$$x(t) = A \sin(\alpha t - I \cos(\beta t + \Phi_m)) \quad (2.7)$$

Fouriersserien blir då istället [2]:

$$x(t) = \sum_{n=-\infty}^{\infty} J_{|n|}(I) \sin\left(\alpha t + n\beta t + n\Phi_m - |n|\frac{\pi}{2}\right) \quad (2.8)$$

Insättning av $\phi_m = \pi/2$ i 2.7 och 2.8 ger ekvation 2.3 resp 2.4. Man kan här se att modulatorfasen inte påverkar sidobandens *amplituder* på något vis, och detta är troligtvis anledningen till att fasparametern tidigare ignorerats. Sidobandens *fas* påverkas däremot i allra högsta grad. Denna är, som vi tidigare har sett, viktig på grund av fenomenet med reflekterade sidoband: när en negativ, reflekterad, frekvenskomponent sammanfaller med en positiv är fasen avgörande för den resulterande komponentens amplitud.

Ovanstående resonemang erbjuder också ett nytt sätt att förklara ett inom FM-syntes välkänt fenomen: när förutsättningarna för interferens är uppfyllda och bärvåg och modulator justeras till ett förhållande som *nästan* ger ett harmoniskt spektrum, kan långsamma cykliska variationer av tonens klangfärg höras. Det konventionella, och helt korrekta, sättet att förklara detta är att par av närliggande frekvenskomponenter bildas. Dessa interfererar med varandra och lågfrekventa svävningar uppstår. Man kan istället tänka sig att en lätt felstämd modulator motsvarar en rättstämd modulator med *kontinuerligt linjärt varierande modulatorfas*, vilket därmed orsakar cykliska variationer av klangfärgen. Dessa två förklaringar är förstås helt likvärdiga rent matematiskt.

2.5 Utvidgningar av grundkonceptet

Det basfall av FM som här beskrivits kan utökas; en ganska uppenbar utvidgning är att använda två modulatorer (Schottstaedts "Complex Modulating Wave" [4]). Avancerade FM-synthar som Yamaha:s DX-serie ännu har fler operatorer; DX7:an har till exempel sex stycken. Dessa kan då kombineras i olika förbestämda konfigurationer. Dessutom finns feedbackloopar insatta i signalvägen (se kapitel 2.6). På vissa synthar kan man också använda sig av andra vågformer än sinus.

Bland de mer udda varianterna finns asymmetrisk FM, AFM [5], där grundformeln modifierats så att dess fouriersserie istället blir

$$x(t) = \sum_{n=-\infty}^{\infty} r^n J_n(I) \sin(\alpha t + n\beta t) \quad (2.9)$$

Här finns en extra faktor r som skapar asymmetri i spektrat. Observera att detta skiljer sig från att med hjälp av ett filter skapa asymmetri, eftersom r har samma inverkan oavsett om frekvenskomponenten reflekteras eller ej. Metoden har

dock inte fått något genombrott - kanske på grund av att den är relativt tung att beräkna (se [6]).

En annan variant som också ger ett asymmetriskt spektrum, men kräver betydligt mindre beräkningskraft, är DFM (Double Frequency Modulation) [7]:

$$x(t) = \sin(I_1 \sin \alpha_1 t + I_2 \sin \alpha_2 t) \quad (2.10)$$

DFM, som i själva verket är ett specialfall av Schottstaedts "Complex Modulating Wave", har inte heller fått någon större betydelse.

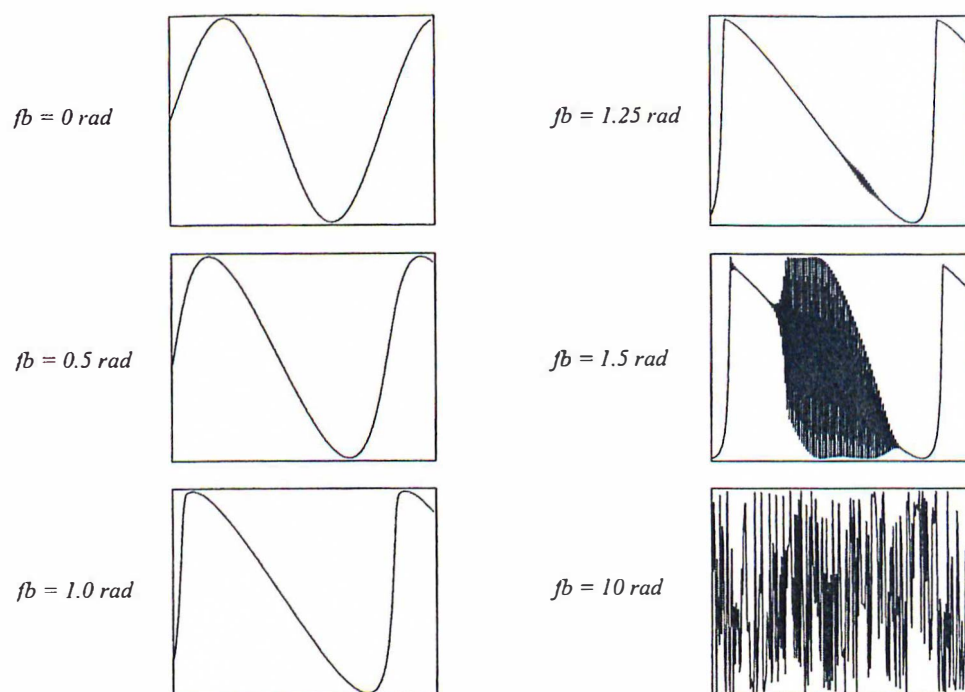
Ofta är det svårt att hitta bra parametrar till en FM-synthesizer - variationsmöjligheterna är närmast oändliga. En del intressanta metoder för hur FM-parametrar kan "matchas" för att uppnå ett visst ljud har utvecklats. Gemensamt för metoderna är att man skapar ett tidsvariant spektrum av det ljud man vill syntetisera. Normalt sett tar man endast hänsyn till harmoniska övertoner, vilket gör att FFT fungerar bra för denna analys (detta gör tyvärr också att metoderna är mindre lämpade för trumljud, som vanligtvis är mer komplicerade än så - speciellt i attacken). Vissa försöker sedan systematiskt anpassa FM-spektrat till denna referens [8], medan andra använder sig av genetiska algoritmer [9, 10]. Det senare verkar ge bäst resultat för FM.

2.6 Återkopplad FM

Genom att *fasmodulera* en oscillator med sin egen utsignal kan man påverka vågformen på ett intressant sätt: allteftersom återkopplingen ökar närmar sig utsignalen en sågtandsvåg i formen. När den ökas ytterligare uppstår högfrekventa oscillationer mitt på vågen. Återkopplar man riktigt mycket uppträder systemet helt kaotiskt - vi har skapat en brusgenerator (Fig. 2.3).

Den "pseudo-sågtand" som kan skapas med feedback innehåller en DC-komponent. Därför kan bärvågens grundfrekvens förflyttas om man modulerar med en feedback-oscillator i en FM-implementation med "riktiga oscillatorer". Feedbackloopar kan även vara långa, så att en oscillator moduleras av en annan längre fram i signalvägen - detta finns till exempel på DX7.

Metoden kan också vara intressant för att skapa en aliasing-fri digital sågtandsvåg.

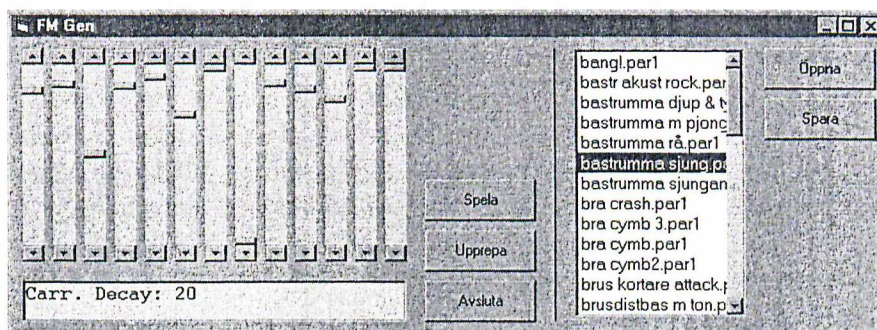


Figur 2.3: Sinusoscillator med feedback: vågformens utseende vid olika grad av återkoppling.

3 Konstruktion av algoritmer

3.1 FM Gen

Syntes av trumljud med hjälp av FM är ett relativt outforskat område (lite finns i Chownings klassiska artikel [1], s. 533). Ett naturligt första steg var därför att skaffa en plattform där olika algoritmer, kända och okända, enkelt kunde testas. Istället för att använda något av de färdiga program som finns för dylika ändamål (till exempel CSound) alternativt något matematiskt verktyg (Matlab, Mathematica...) skrevs ett speciellt program i Visual Basic; *FM Gen* (fig. 3.1). En egenutvecklad programvara ger full kontroll över hur alla beräkningar utförs, vilket underlättade när algoritmerna slutligen skulle implementeras i DSP-assembler. Det är även mer obegränsat och flexibelt, och man kan också vara säker på att de ljud man finner verkligen kan återskapas exakt i DSP:n. *FM Gen* fungerar som ett "ramverk" där den ljudgenererande koden enkelt kan ändras och bytas ut. Programmet tillhandahåller ett antal "sliders" som kopplas till lämpliga parametrar i algoritmerna. Vid ett tryck på "Spela"-knappen förberäknas ljudet i en buffer och spelas sedan upp. Möjlighet finns även att spara intressanta inställningar som man hittar.



Figur 3.1: *FM Gen*

Ett exempel på hur den enklaste möjliga FM-generatorn kan se ut i VB:

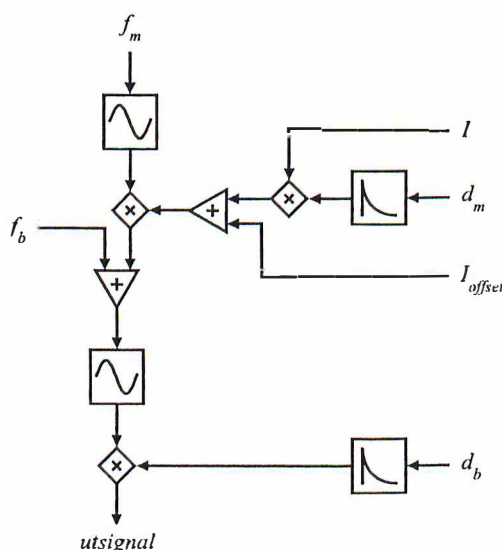
```
For I& = 1 To NSamples
    WaveData!(I&) = SinOsc1(CarrierFreq! + ModulatorIndex!
        * SinOsc2(ModulatorFreq!))
Next
```

Med de funktioner och variabler som *FM Gen* tillhandahåller går det lätt att prova olika algoritmer och programmet visade sig vara ett mycket värdefullt verktyg i sökandet efter intressanta ljud.

3.2 Utvecklingsarbetet

Som utgångspunkt valdes den enkla 2-operators FM-modell som Chowning beskriver [1] (fig. 3.2). Denna modifierades och utökades på olika sätt. Vissa varianter visade sig vara oanvändbara i sammanhanget, medan andra faktiskt kunde skapa trumliknande ljud. Försök gjordes även med DFM och AFM, men ingen av dessa gav särskilt användbara resultat.

Chownings modell innehåller 2 operatorer; en modulator och en bärvåg. Den innefattar även två *enveloppgeneratorer*. En enveloppgenerator skapar en signal som varierar med tiden på ett visst sätt; enveloppen. Denna signal kan exempelvis användas för att styra en operators amplitud. Envelopper kan se olika ut beroende på vad de används till - lämplig volymenvelop till ett pianoljud är en långsamt sjunkande signal, medan trumljud oftast har en snabb envelopp. I den aktuella modellen finns en envelopp för volymen (bärvågsamplituden) och en för modulatoramplituden. Modulatorenveloppen innebär att mängden modulation varierar över tiden och det "dynamiska spektrum" som berördes i inledningen kan därmed ådstakommas. En offset adderas till enveloppen, så att ljudet kan "landa" på ett visst modulationsdjup. Chowning använder s.k. ADSR¹-envelopper, men rekommenderar en exponentiellt avklingande volymkaraktäristik för trummor.

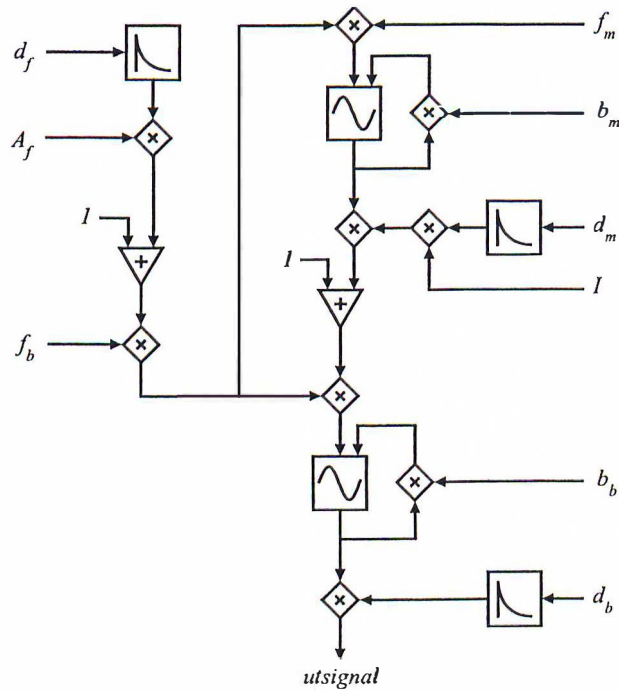


Figur 3.2: Chownings modell (se kapitel 2.3 för detaljer angående implementeringen).

Efter en del experimenterande hittades med utgångspunkt i den ovan beskrivna modellen en utökad variant som gav en hel del intressanta resultat. Denna fick bli den nya "språngbrädan", och alla de algoritmer som slutligen togs fram, med undantag för cymbalen, bygger på den (fig. 3.3). Oscillatorblocken symboliserar här tabelloscillatorer (se kapitel 2.3), där den momentana frekvensen kan styras. Modulatorns frekvens är kopplad till bärvågens, så att samma förhållande alltid råder mellan dem. Även moduleringsdjupet (mängden modulation) styrs av bärvågens frekvens - annars minskar moduleringen när frekvensen ökar. Återkopplingsslingor finns på både modulator och bärvåg. Den offset som adderas till modulatorenveloppen i figur 3.2 togs bort. Slutligen finns en tredje exponentiellt avklingande envelopp som styr ljudets grundfrekvens - något som är användbart vid syntes av basdrum. Ursprungligen fanns även en fjärde envelopp som styrde modulatorfrekvensen. Detta ovanliga arrangemang gav udda effekter, men fick aldrig rum i någon av de slutgiltiga implementeringarna.

1. "Attack, Decay, Sustain, Release" - det vanligaste sättet att kontrollera en envelopps utseende.

Den ovan beskrivna modellen kan med rätt parameterar ge förvånansvärt "akustiska" resultat. Mycket återkoppling på modulatorens (ger brusiga toner) och en kort modulatorenvelop kan ibland återge anslaget på en puka relativt realistiskt. Kombinerat med frekvensenveloppen blir resultatet ännu bättre: ett kort svep i början gör anslaget "snärtigare". Det kan också vara bra att låta oscillatorerna starta en bit in i perioden, och på så sätt skapa ett "klick" precis i början av ljudet.



Figur 3.3: Grundmodellen

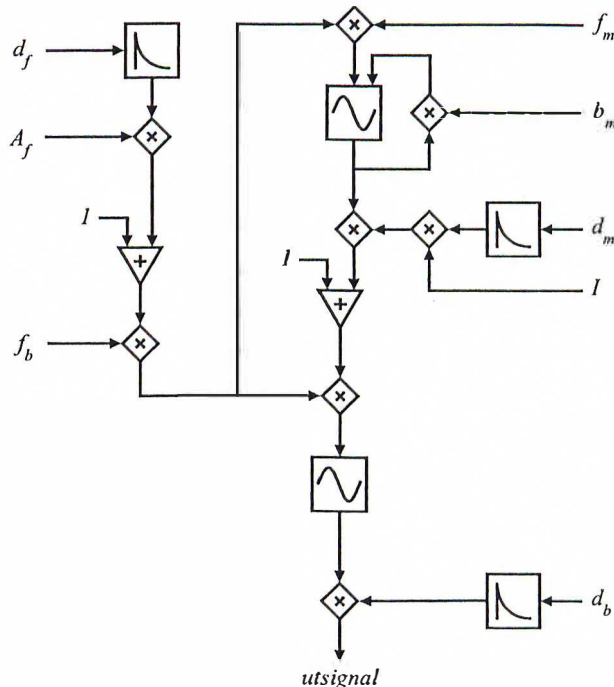
4 Resultat

4.1 Bastrumma

Denna modell består av grundmodellen minus återkoppling på bärvågen (fig. 4.1). Rattarna på Machinedrum är kopplade till följande parametrar:

- Frekvens (f_b)
- Decay (utklingningstid för volymenveloppen) (d_b)
- Modulatorfrekvens (f_m)
- Modulationsdjup (I)
- Modulatordecay (d_m)
- Modulatoråterkoppling (b_m)
- Amplitud för frekvensenveloppen (A_f)
- Decay för frekvensenveloppen (d_f)

För ett mer distinkt anslag startas oscillatorerna med en fasförskjutning på $\pi/2$ rad (dvs. cosinus). Det skarpa steget i början av vågformen skapar ett högfrekvent "klickljud" som påminner lite om när hammaren träffar trumskinnen. Mest realistiska ljud får man när modulatoren återkopplas så att den brusar och modulatorenaveloppen är relativt snabb. Annars kan modellen varieras i oändlighet och skapa alla möjliga underliga ljud.



Figur 4.1: Bastrumma

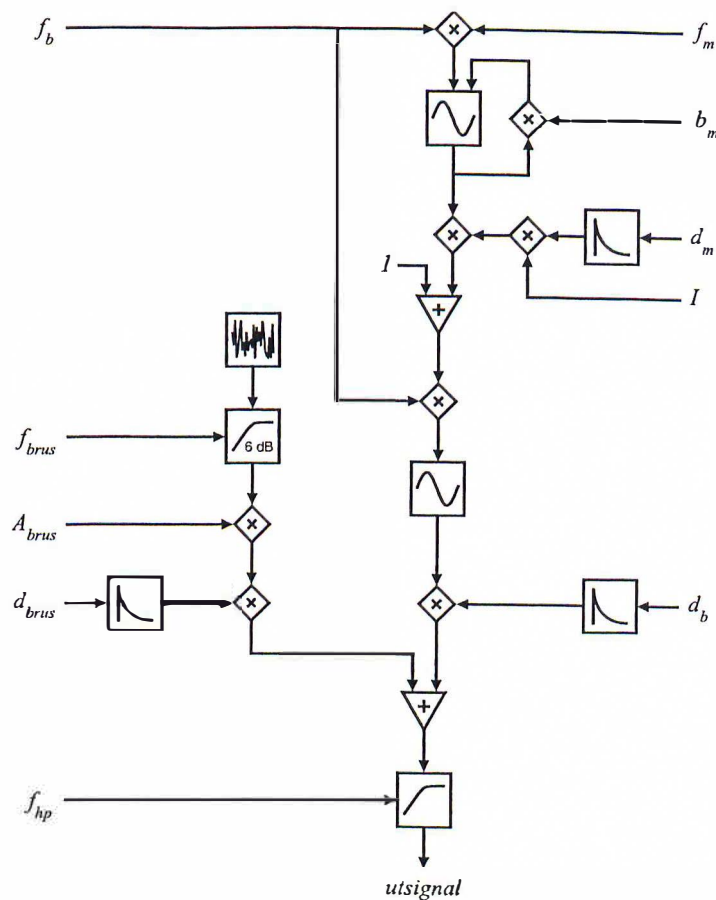
4.2 Virveltrumma

Med standardmodellen kan någorlunda bra virvelljud skapas. Ungefär samma inställningar som ger en bra bastrumma kan användas för virvel, om den stäms högre. För att få ett bra ljud krävs dock att man även återskapar ljudet av sejarren; metallbandet som ligger spänt mot underskinnet på en virveltrumma. Detta arrangemang skapar ett brusande ljud när trumman slås an och är den viktigaste anledningen till att virveln skiljer sig så mycket från en vanlig puka. Denna modell använder en brusgenerator följd av ett högpasfilter - de lägre frekvenserna är inte önskvärda i detta sammanhang. Man kan också använda ett bandpassfilter, så att bruset inte blir för ljust eller vasst, men resultatet var tillfredställande med enbart högpasfiltrering.

Frekvensenveloppen som finns i standardmodellen är borttagen - antalet parametrar måste begränsas och den är inte så användbar vid syntes av virvelljud. Sist i signalkedjan ligger ännu ett högpasfilter - något som är bra ifall ljudet är för dovt eller "burkigt". Följande parametrar kan kontrolleras av användaren:

- *Frekvens (f_b)*
- *Decay (d_b)*
- *Modulatorfrekvens (f_m)*
- *Modulationsdjup (I)*
- *Modulatordecay (d_m)*
- *Brusnivå (A_{brus})*
- *Amplitud för frekvensenveloppen (d_{brus})*
- *Brytfrekvens för högpasfiltret (f_{hp})*

Modulatorns återkoppling (b_m) är låst på ett högt värde (ger mycket brus). Även högpasfiltret efter brusgeneratoren är låst till ett fast värde (f_{brus}).



Figur 4.2: Virveltrumma

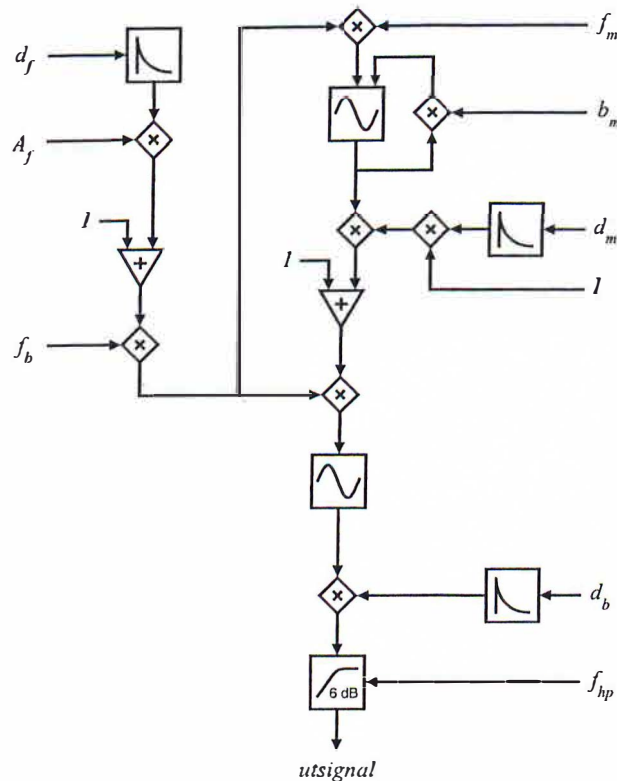
4.3 Pukor

Samma modell som används för bastrumman (fig. 4.1) kan med fördel användas för pukor - allt som krävs är att frekvensen justeras lite högre. Ljuden har dock en tendens att blir ganska "mullriga" och dova. Därför finns ett högpasfilter inlagt sist i signalkedjan. Dess brytfrekvens kopplas till grundtonens och kan inte kontrolleras på annat vis av användaren. Möjlighet finns också att styra oscillatorernas initiala fas och därmed variera mängden "klick" i början av ljudet. Kontroller:

- Frekvens (f_b)
- Decay (d_b)
- Modulatorfrekvens (f_m)
- Modulationsdjup (I)
- Modulatordecay (d_m)
- Amplitud för frekvensenveloppen (A_f)
- Decay för frekvensenveloppen (d_f)

- Startfas (varieras mellan 0 och $\pi/2$)

Modulatoråterkopplingen (b_m) är låst till ett högt värde.



Figur 4.3: Puka

4.4 Handklapp

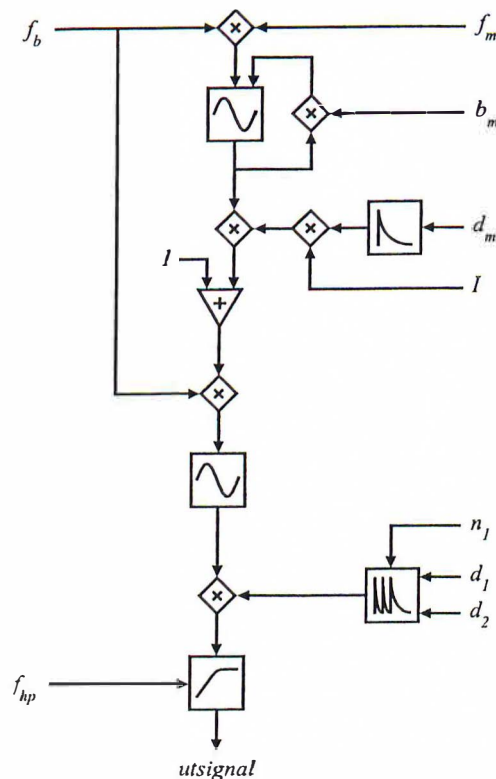
Ett klappliknande ljud kan skapas med standardmodellen: Modulatorn återkopplas så att den brusar och modulationsdjupet sätts ganska högt. Detta resulterar i något som liknar vitt brus med en resonanstopp, vilket i kombination med en snabb volymenvelopp och lite fantasi ger ett klappliknande ljud. Vad man egentligen vill åstadkomma är dock ljudet av flera människor som klappar samtidigt. Ett vanligt sätt att återskapa denna effekt är att använda en speciell volymenvelopp, bestående av ett antal snabba "förklapp" innan den slutgiltiga utklingningen. Detta ger ett intryck av flera samtidiga klapp som är aningen "osynkade". Enveloppen fungerar i detta fall så att den triggar om när den når ner till 10% av maximal amplitud - detta upprepas tills det är dags för den sista, längre, utklingningen.

Sist ligger även i denna modell ett högpasfilter - ljudet kan ofta blir för dovt annars. Följande parametrar kan kontrolleras:

- Frekvens (f_b)
- Modulatorfrekvens (f_m)
- Modulationsdjup (I)
- Modulatordecay (d_m)

- Förklappsdecay (d_1)
- Decay (d_2)
- Antal förklapp (n)
- Brytfrekvens för högpasfiltret (f_{hp})

Modulatorns återkoppling (b_m) är låst på ett högt värde. Figur 4.4 beskriver handklapp.



Figur 4.4: Handklapp

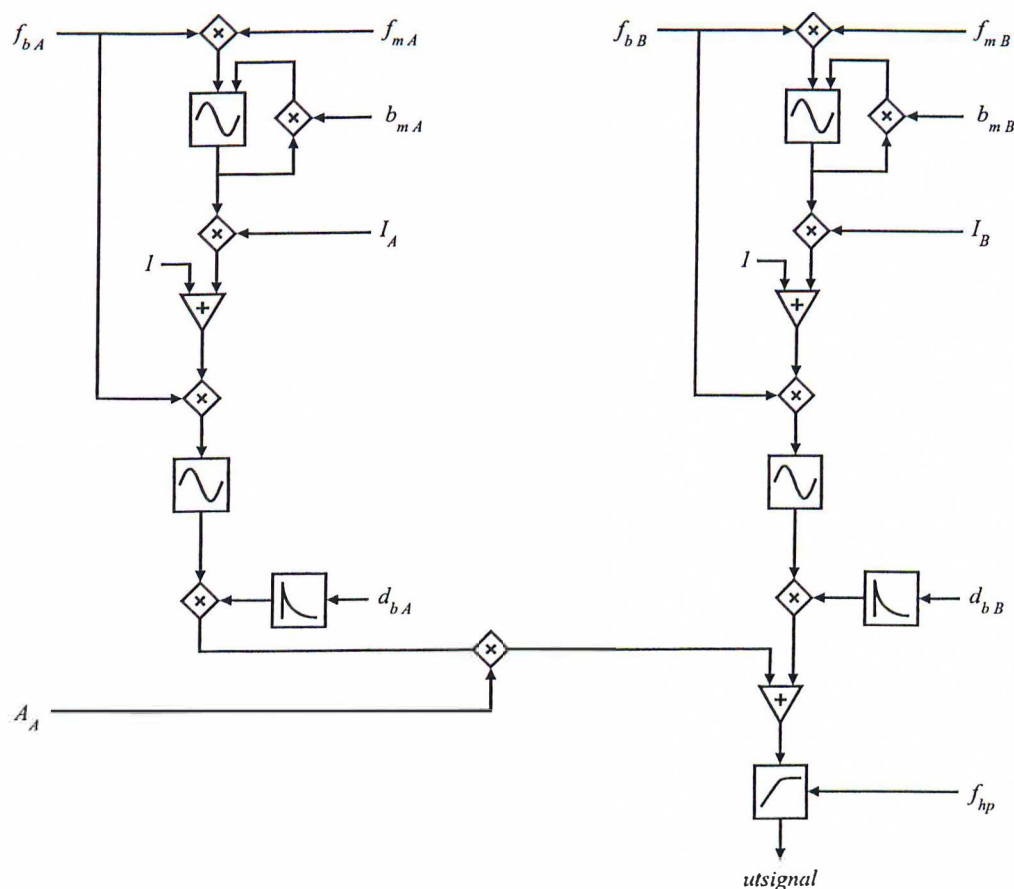
4.5 Kantslag

Detta ljud kan sägas bestå av (minst) två komponenter; trumstocken som träffar virveltrummans kant och resonansen i själva trumman. Det förstnämnda skapas på samma sätt som klappljudet (se föregående avsnitt) - modulationsljudet sätts lämpligtvis aningen lägre, vilket ger mer ton i ljudet. Virvelljudet skapas på samma sätt som i virvelmodellen, dock utan brusgenerator. Dessutom har modulatorens enveloppen bortrationaliserats. Sist ligger det vanliga högpasfiltret (fig. 4.5). Kontroller:

- Frekvens, kantslag ($f_{b\ B}$)
- Decay, kantslag ($d_{b\ B}$)
- Modulationsdjup, kantslag (I_B)
- Frekvens, virvel ($f_{b\ A}$)
- Decay, virvel ($d_{b\ A}$)

- Modulationsdjup, virvel (I_A)
- Virvelljudets volym (A_A)
- Modulatordecay (d_m)
- Brytfrekvens för högpasstret (f_{hp})

Modulatorernas återkoppling ($b_{m A}$, $b_{m B}$) är låst på höga värden. Även de relativa modulatorfrekvenserna ($f_{m A}$, $f_{m B}$) är fasta.



Figur 4.5: Kantslag

4.6 Koklocka

Detta instrument har en speciell karaktär - lite falsk eller ostämd på något vis. Rolands gamla analoga trummaskin TR-808 använder två disharmoniskt stäm-
da fyrkantsvågsoscillatorer, och resultatet måste anses väldigt lyckat: ljudet,
som egentligen inte är särskilt likt det akustiska originalet, har en väldigt karak-
tärisk och behaglig klang och frågan är om det inte har passerat sin förebild i
popularitet¹.

För att göra det enkelt togs TR-808:s som utgångspunkt. Istället för fyrkantsvå-
gor användes två FM-par med återkoppling på bärvågen - på så sätt kan något

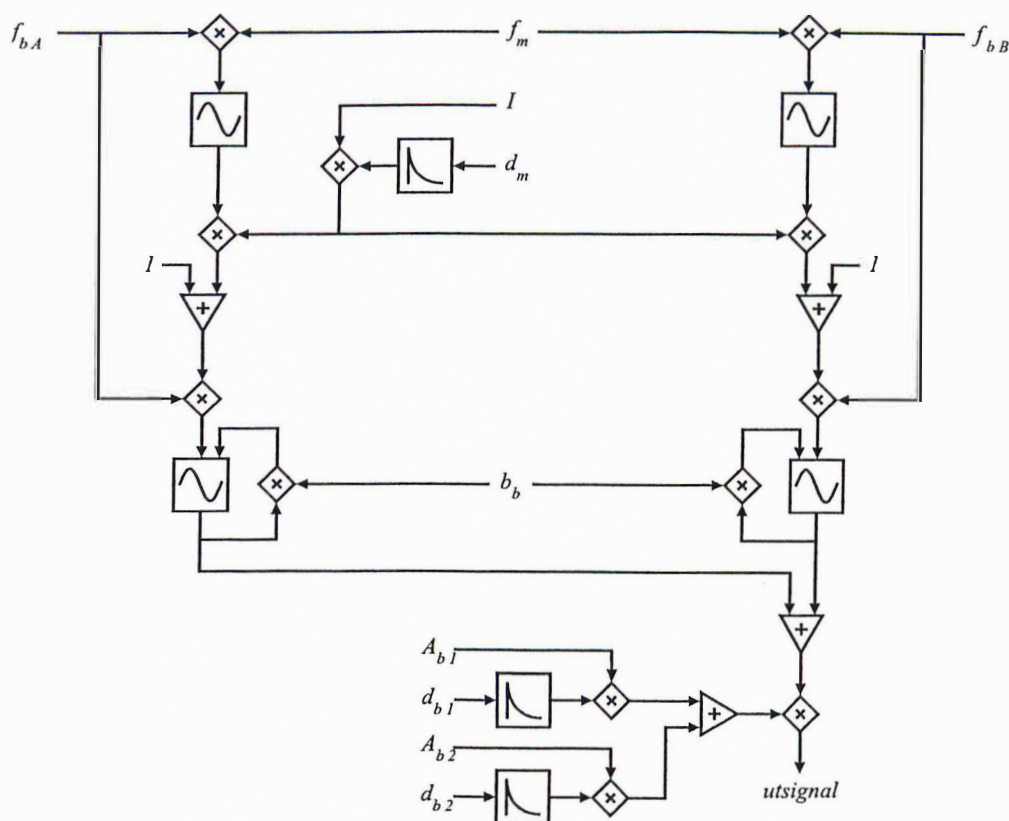
1. Lyssna på introet till Whitney Houstons "I wanna dance with somebody" - du känner säkert igen lju-
det från många andra låtar.

som påminner om fyrkantsvågens skarpa ljud skapas (se kapitel 2.6). I TR-808 har den ena oscillatoren en frekvens som är ca. 48% högre än den andra, och de två FM-paren låstes därför i detta förhållande.

Volymenveloppen gjorts lite annorlunda: en snabb och en långsam adderas och resultatet blir ett skarpare anslag med bibehållen utklingningstid. De båda modulatorerna delar samma envelopp - två separata ger bara onödigt mycket kontrollmöjligheter. Det samma gäller återkopplingen.

- Frekvens ($f_{bA}, f_{bB} = f_{bA} * 1.48$)
- Decay (utklingningstid för volymenveloppen) (d_{b1})
- Modulatorfrekvens (f_m)
- Modulationsdjup (I)
- Modulatordecay (d_m)
- Bärståsåterkoppling (b_b)
- "Skarphet" (A_{b2})

d_{b2} beräknas så att den alltid är betydligt kortare än d_{b1} . $A_{b1} = 1 - A_{b2}$. Figur 4.6 beskriver algoritmen.



Figur 4.6: Koklocka

4.7 Cymbaler (öppen och stängd hihat, ride, crash)

Att försöka syntetisera cymballjud utgör en stor utmaning; ljuden är såpass invecklade att de blir svåra att efterlikna med enkla "analoga" modeller. De skall

ha ett brett spektrum med mängder av frekvenskomponenter men ändå inte vara alltför brusliknande. Ännu en gång sneglades lite på Roland TR-808, där en lösning liknande koklockans med framgång används - i det här fallet sex fyrkantsvågor med inbördes dissonant stämning.

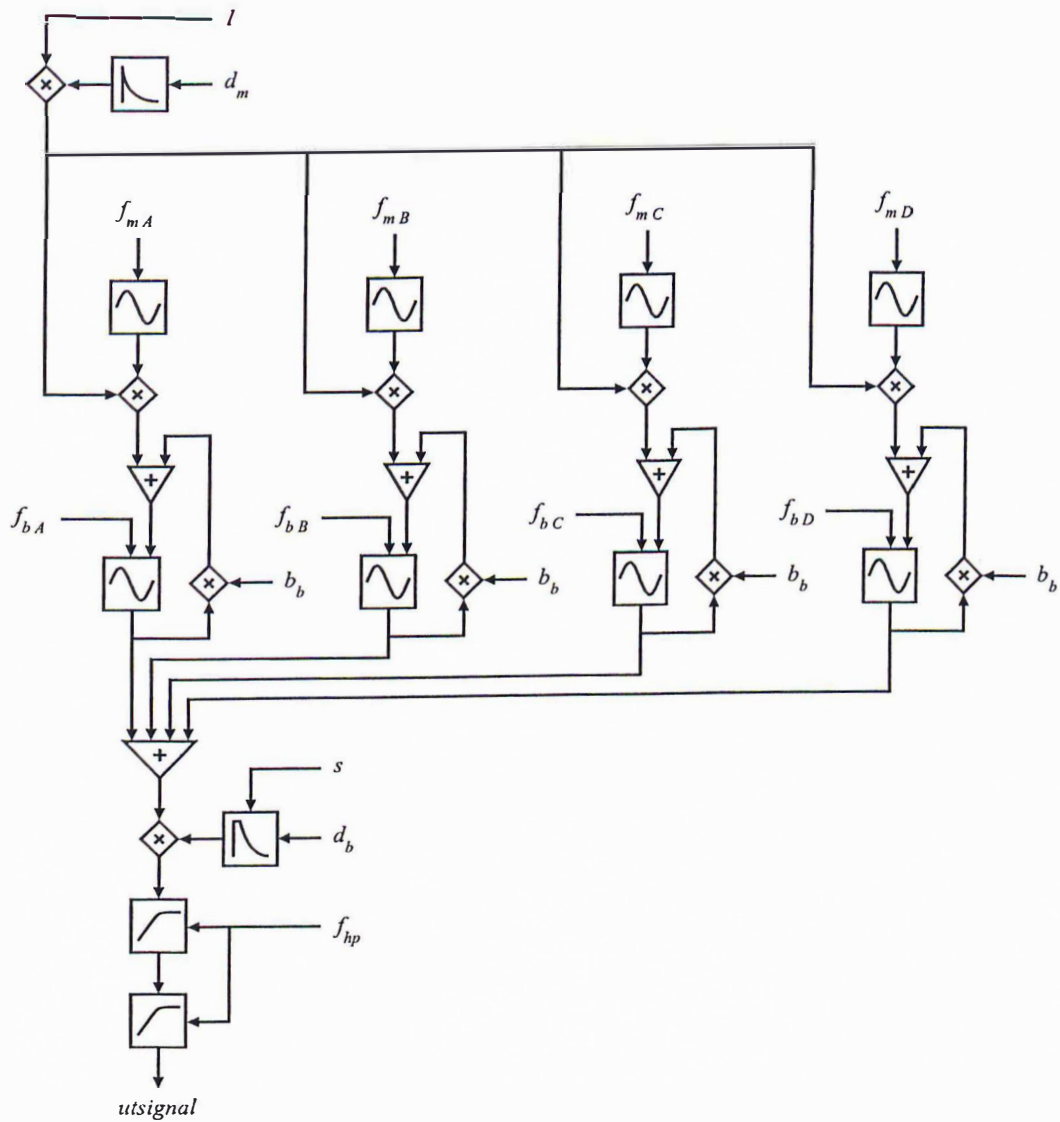
Eftersom FM är ganska tungt att beräkna är sex stycken FM-par, dvs. 12 operatörer, i detta sammanhang helt uteslutet om realtidskraven skall kunna mötas (max. ca 130 maskinkodsinstruktioner/sample). Fyra par med inbördes stämning tagen från 808:an fick räcka. För att ytterligare öka effektiviteten används "fasmodulering" enligt Chownings grundformel (ekvation 2.1) - detta spar några cykler.

Den vanliga exponentiella enveloppen kan ibland kännas lite väl distinkt och skarp på en cymbal - en öppen hihat kan till exempel vara ganska mjuk. Därför finns möjligheten att låta amplituden ligga konstant en stund i början. Detta sker i praktiken genom att multiplicera enveloppens värde med en faktor och låta resultatet begränsas (kan ske automatiskt i 56303-processorn).

Det vanliga högpasfiltret används, men här två gånger i serie - det lät helt enkelt bättre (mer om filtret i kapitel 5.6). Användaren har följande kontrollmöjligheter:

- *Frekvens* ($f_{bA}, f_{bB} = f_{bA} * 1.411, f_{bC} = f_{bA} * 1.8, f_{bD} = f_{bA} * 2.7$)
- *Decay* (utklingningstid för volymenveloppen) (d_b)
- *Modulatorfrekvens* ($f_{mA}, f_{mB} = f_{mA} * 1.411, f_{mC} = f_{mA} * 1.8, f_{mD} = f_{mA} * 2.7$)
- *Modulationsdjup* (I)
- *Modulatordecay* (d_m)
- *Bärvågsåterkoppling* (b_b)
- *Mättnad* för volymenveloppen (s)
- *Brytfrekvens* för högpasfiltret (f_{hp})

Den ovan beskrivna modellen (se fig. 4.7) används till samtliga cymbaler (crash, ride, hihat).



Figur 4.7: Cymbal

5 Implementering

5.1 Hårdvaran

Ljudgenerering i Machinedrum sker med hjälp av en Motorola 56303 DSP, klockad på 100 MHz. Denna processor skall kunna beräkna 16 olika trumljud samtidigt, med samplingsfrekvensen 44.1 kHz (innebär max ca. 130 instruktioner/sample). Ytterligare en 56303 sköter mixning av de 16 kanalerna och effekter (bl.a. lågpas-, högpasfilter och ringmodulering för varje kanal samt EQ, delay, reverb och kompressor på mixen). Operativsystemet körs på en 40 MHz Motorola 5206 Coldfire-processor som kommunicerar med DSP-processorerna via DMA. Dess arbetsuppgifter innefattar bl.a. stepsequencer¹, LFO:er², tolkning av MIDI³-meddelanden och hantering av det grafiska gränssnittet. För det sistnämnda ändamålet finns en 128 x 64 pixels grafisk LCD. 16 tryckknappar bildar en stepsequencer och för justering av trumljudens karaktär finns åtta "oändliga" pulsgivar-rattar.

I anslutningsväg finns MIDI in, ut och thru, stereomix ut L/R, 4 st. konfigurerbara audioutgångar och två st. audio in.

5.2 DSP-programmering

Samtliga trummodeller som beskrivs i kapitel 4 har implementerats i DSP-assembler. Kraven på kodens effektivitet var som tidigare nämnts höga och därför var det nödvändigt att utnyttja 56303:s speciella egenskaper så mycket som möjligt. Dessa innefattar bl.a. möjligheten att förflytta saker i de två interna minnena parallellt med andra instruktioner.

5.3 Coldfire-programmering

Användaren av Machinedrum har som tidigare nämnts möjligheten att kontrollera ljudet på varje trumma med hjälp av åtta rattar. Rattarnas värden måste omvandlas i operativsystemet till lämpliga parametervärden innan de skickas vidare till den ljudgenererande DSP:n. Till varje trummodell finns därför C-kod som tar hand om saker som till exempel logaritmisk respons på frekvensratten.

5.4 Brusgeneratören

En digital brusgenerator är egentligen inget annat än en slumpalsgenerator. Det finns många sätt att generera slumpal, eller snarare *pseudo*-slumpal (äkt är betydligt svårare). Här användes den välkända Fibonacci-sekvensen, där nästa tal i serien är summan av de två föregående:

$$\begin{aligned} f_0 &= a \\ f_1 &= b \\ f_n &= f_{n-1} + f_{n-2} \end{aligned}$$

1. Ett enkelt sätt att komponera trumrytmer; varje takt delas in i 16 steg. Varje steg har en egen knapp på trummaskinen. Genom att trycka på en knapp aktiverar man en triggnings på motsvarande tidpunkt. Detta görs individuellt för varje kanal.
2. "Low Frequency Oscillator": långsamma oscillatorer som kan kopplas till olika parametrar i en synt och på så sätt skapa intressanta variationer i ljudet.
3. "Musical Instruments Digital Interface": seriellt gränssnitt för kommunikation mellan elektroniska musikinstrument.

a och b är slumtalsfrön. Kombinerat med den modulo-räkning som addition i processorregistren medför blir detta en väl fungerande brusgenerator. a och b kan väljas godtyckligt så länge inte båda är 0. De bör dock inte vara för små, eftersom man då får ett insvängningsförlopp.

5.5 Oscillatorer

Oscillatorerna implementerades med hjälp av en sinustabell, i detta fall bestående av $2^{15} = 32768$ 24-bitars värden. Med 56303-processorn kan indexering i en sådan tabell ske väldigt effektivt eftersom de speciella indexregistren kan fås att stega fram automatiskt efter varje minnesaccess. Till varje indexregister finns också ett moduloregister, som kan användas för automatisk "wrapping" av indexpekaren.

För att uppnå bästa möjliga signal/brus-förhållande skall tabellen egentligen vara betydligt större - en sinustabell med 24-bitarstal bör bestå av $2^{25}/2\pi$ värden. Då är man säker på att ett steg framåt eller bakåt aldrig innebär en förändring på mer än 1 LSB. En sådan tabell tar dock mycket plats i minnet. Ett alternativ är att minska tabellen och interpolera mellan värdena. Detta kan spara mycket plats till priset av små, ofta nästan försumbara, försämringar av signalkvaliteten. Problemet är att interpoleringen tar värdefull beräkningskraft och jag nöjde mig därför med den tidigare beskrivna lösningen - den fungerade tillräckligt bra.

5.6 Högpasfilteret

Kraven på filtret var inte särskilt höga - tanken var bara att möjligheten skulle finnas att sänka de lägre frekvenserna en aning. Ett enkelt första ordningens högpasfilter visade sig fungera utmärkt (ekvation 5.1).

$$H(z) = \frac{1 - z^{-1}}{1 - az^{-1}} \quad (5.1)$$

Utifrån $H(z)$ får vi följande differensekvation:

$$y(n) = x(n) - x(n-1) + ay(n-1) \quad (5.2)$$

Ekvation 5.2 kan enkelt implementeras i DSP-assembler.

6 Slutsats och kommentar

Sammanfattningsvis kan FM-algoritmen sägas vara förvånansvärt användbar när det gäller syntes av trum- och percussionljud. Även om realism inte var högsta prioritet i detta arbete är många av de resulterande algoritmerna kapabla att producera relativt verklighetstroga ljud.

De uppgifter som ingick i examensarbetet är lösta, men givetvis skulle man kunna göra mycket mer arbete inom detta område. Intressant hade varit att försöka utveckla och anpassa de metoder som finns för automatisk parameterbestämning så att de även fungerar för trumljud. Sofistikerade algoritmer som bygger på bl.a. genetisk programmering har utvecklats för FM [8, 9, 10], men de förutsätter att det ljud som skall efterliknas är av mer harmonisk karaktär än vad trumljud ofta är.

Referenser

- [1] J. M. Chowning, "The Synthesis of Complex Audio Spectra by Means of Frequency Modulation", *Journal of the Audio Engineering Society*, vol. 21, nr. 7, pp. 526-534 (1973).
- [2] J. A. Bate, "The Effect of Modulator Phase on Timbres in FM Synthesis", *Computer Music Journal*, vol. 14, nr. 3, pp. 38-45 (1990)
- [3] F. Holm, "Understanding FM Implementations: A Call for Common Standards", *Computer Music Journal*, vol. 16, nr. 1, pp. 34-42 (1992)
- [4] B. Schottstaedt, "The Simulation of Natural Instrument Tones Using Frequency Modulation with a Complex Modulating Wave", *Journal of the Audio Engineering Society*, vol. 1, pp. 46-50 (1977).
- [5] J.-P. Palamin, P. Palamin, "A Method of Generating and Controlling Musical Asymmetrical Spectra", *Journal of the Audio Engineering Society*, vol. 36, nr. 9, pp. 671-685 (1988)
- [6] B.T. G. Tan, S. L. Gan, "Real-Time Implementation of Asymmetrical Frequency Modulation Synthesis", *Journal of the Audio Engineering Society*, vol. 41, no. 5, pp. 357-363 (1993)
- [7] B.T. G. Tan, S. L. Gan, S. M. Lim, S. H. Tang, "Real-Time Implementation of Double Frequency Modulation (DFM) Synthesis", *Journal of the Audio Engineering Society*, vol. 42, nr. 11, pp. 918-926 (1994)
- [8] J. W. Beauchamp, "Synthesis by Spectral Amplitude and Brightness Matching of Analyzed Musical Instrument Tones", *Journal of the Audio Engineering Society*, vol. 30, nr. 6, pp. 396-406 (1982)
- [9] A. Horner, J. Beauchamp, L. Haken, "Machine Tongues XVI: Genetic Algorithms and Their Application to FM Matching Synthesis", *Computer Music Journal*, vol. 17, nr. 4, pp. 17-29 (1993)
- [10] A. Horner, "Nested Modulator and Feedback FM Matching of Instrument Tones", *IEEE Transactions on Speech and Audio Processing*, vol. 6, nr. 4, pp. 398-409 (1998)
- [11] Manualen till Yamaha DX7-II